

E-PAYMENT MECHANISM : E-WALLETS AND UNIFIED PAYMENTS INTERFACE

**Bachelor of Technology
Computer Science and Engineering (AIML)**

Submitted By

ANIKET DATTA (13030822055)

MARCH 2025



**Techno Main
EM-4/1, Sector-V, Salt Lake
Kolkata- 700091
West Bengal
India**

ABSTRACT:

Digital Public Infrastructure enables the delivery of services, facilitating exchange of data and supporting digital governance across various sectors. Digital Public Infrastructure includes elements such as digital identity systems, Payment Platforms, E-Wallets, Unified Payments Interface and so on. The Data Exchange Protocols are to be efficient, scalable, engineered, interoperable and accessible to both government and private sector participants. These infrastructures aim at supporting the functioning of public services, governance and economic processes. This can be understood as an intermediate layer in the digital ecosystem enabling applications across various public sectors. Some systems like the AADHAR identification System is a part of digital public infrastructure, Unified Payments Interface for payments, and India Stack Data Exchange framework. X-Road is such an open-source government data exchange system. These also play a major role in areas like electronic governance, health records, management, and education. To be effective, DPI implementation must carefully balance privacy safeguards, data protection, and inclusive accessibility for all population segments. The conceptualization of digital systems as public infrastructure has evolved from historically siloed, publicly-funded technologies like the Internet and GPS. Early DPI models include India's Aadhaar biometric identity system and the India Stack—government-owned, open-standard digital utilities for public and private service delivery—and Estonia's X-Road, a secure data exchange layer for interoperable e-government services. By the 2020s, this infrastructure analogy was formalized, culminating in a 2023 G20 consensus defining DPI as interoperable, open-standard systems for equitable, societal-scale service delivery, a framework subsequently endorsed by the UN for advancing Sustainable Development Goals. The rapid expansion of digital finance is largely driven by the development of electronic payment systems, notably e-wallets and the Unified Payments Interface (UPI). This analysis compares the technical architecture, operational models, and broader societal influence of these two systems. E-wallets operate as stored-value platforms within controlled ecosystems, necessitating advance loading of funds and functioning through exclusive merchant agreements. Conversely, UPI is an open-standard, interoperable protocol that enables immediate bank-to-bank transactions without requiring a dedicated stored-value account, effectively disintermediating the payment process. The evaluation concludes that although e-wallets provide a streamlined user experience within walled gardens, UPI's open-access design and minimal cost framework have accelerated its widespread adoption and enhanced financial inclusion, especially in emerging markets. The dynamic between these competing yet complementary models is transforming financial services, necessitating careful deliberation by regulators and industry stakeholders concerning security, data protection, and the transition to digital economies.

INTRODUCTION:

An **electronic payment (e-payment) mechanism** is a structured protocol and software architecture that facilitates the secure transfer of monetary value over digital networks. This is not merely the front-end application but the entire backend ecosystem of rules, cryptographic processes, and communication frameworks that authenticate, authorize, and settle transactions between parties. At the core, an e-payment system is a trusted intermediary that manipulates and transmits data representing financial liabilities. Its technical operation can be decomposed into several critical layers:

1. **Protocol & Message Layer:** This defines the strict syntax and semantics of communication between entities (e.g., merchant, payer, bank, payment network). Standards like ISO 20022 govern these message formats, ensuring interoperability. Messages contain encrypted instructions for debit/credit actions, transaction IDs, and participant details.
2. **Security & Cryptography Layer:** This is the foundation of trust and employs:
 - **Asymmetric Encryption (Public Key Infrastructure - PKI):** Used for secure key exchange and digital signatures to verify the authenticity and integrity of every message, ensuring non-repudiation.
 - **Symmetric Encryption:** Secures the payload of the transaction data during transmission (e.g., using AES algorithms).
 - **Tokenization:** A critical process where sensitive Primary Account Numbers (PANs) are replaced with algorithmically-generated, non-sensitive tokens. The actual card data is stored in a highly secure, centralized token vault, drastically reducing the risk of data compromise during a transaction.
3. **Ledger & Settlement Layer:** The mechanism interfaces with underlying financial ledgers. It does not store value itself but instructs legacy systems (like core banking systems) to debit and credit the appropriate accounts. Settlement, the final transfer of funds between financial institutions, often occurs in batched processes through central banks or clearinghouses at the end of a cycle.
4. **API Layer:** Modern systems, like UPI, expose a set of well-defined Application Programming Interfaces (APIs). This allows third-party applications (e.g., merchant apps, banking apps) to initiate payments, check status, and handle exceptions by calling these APIs, which then trigger the complex backend processes.

In essence, an e-payment mechanism is a complex distributed system that translates a financial obligation into a secure, standardized data packet, routes it through a validated network of trust, and ultimately triggers an update to financial ledgers, all while guaranteeing confidentiality, integrity, and availability.

E-wallets: An e-wallet (electronic wallet) is a client-server software application that functions as a digitized, stored-value instrument. The core technical operation of e-wallet involves the creation and management of a centralized, pre-funded monetary ledger for a user, abstracting away the need for direct, real-time interaction with traditional banking infrastructure for each transaction.

Technically, an e-wallet's architecture is characterized by the following components and processes:

1. **Stored-Value Account (The Ledger):** Fundamentally, an e-wallet is a private ledger. When a user "adds money," they are initiating a funds transfer (e.g., via net banking, card payment) *into* the wallet provider's pooled account. This action credits the user's specific ledger balance within the wallet's proprietary database. The subsequent transaction occurs against this internal ledger, not the user's bank account directly.
2. **Closed-Loop vs. Open-Loop Ecosystems:**
 - 🟡 Closed-Loop: Operates within a strictly defined merchant network (e.g., Amazon Pay, Starbucks app). Value is locked within the provider's ecosystem, fostering vendor loyalty but limiting utility. Transactions are simple database updates decrementing the user's ledger and crediting the merchant's ledger within the same system.
 - 🟡 Semi-Closed Loop: Allows payments to a wider set of enrolled merchants but still requires settlement between the wallet provider and each merchant outside traditional card networks. This necessitates complex bilateral agreements.
3. **Tokenization as a Security Primitive:** To secure transactions, e-wallets heavily rely on tokenization. The wallet application on the user's device (the client) does not transmit primary account numbers (PANs). Instead, it uses a token—a cryptographically generated surrogate value—to represent the funding instrument. For closed-loop systems, this token is often meaningful only within the provider's own domain.
4. **Client-Server Model with API Mediation:** The user interacts with a client application (mobile app, web portal). Every transaction initiation (payment, recharge) is an API call from the client to the wallet provider's secure servers. These servers authenticate the request, verify ledger balances, apply business logic, and return a confirmation, all within milliseconds.
5. **Settlement Process:** The settlement between the wallet provider and its partnered merchants is a separate, often batched, back-end process. The provider aggregates all transactions and initiates a bulk net transfer to the merchant's bank account after deducting fees, which is distinct from the real-time user experience of payment.

In summary, an e-wallet is a technologically sophisticated form of digital script. It creates a walled garden of financial value by monetizing user float and simplifying the payment experience at the cost of operating within a controlled, proprietary ecosystem rather than on an open, interoperable network. Its efficiency derives from substituting real-time bank settlement with near-instantaneous updates to its own internal ledger.

Unified Payments Interface:

The **Unified Payments Interface (UPI)** is an open-source, interoperable, and real-time payment system architected as a **digital public infrastructure (DPI)**. It is not a product but a protocol—a set of standardized APIs and technical specifications that facilitates instant fund transfers directly between bank accounts through a single digital gateway.

Technically, UPI's operation is defined by its layered, API-driven architecture:

1. **Interoperable Protocol Layer:** UPI's core innovation is its standardized communication layer, governed by the National Payments Corporation of India (NPCI). This layer defines the message format (e.g., for payment initiation, collection requests) that all participating banks and third-party applications (TPAPs) must adhere to. This ensures any UPI client app can interact with any participating bank.
2. **Decentralized Authentication & Authorization:** Unlike an e-wallet's centralized ledger, UPI is a decentralized system. It acts as a routing and messaging platform. It does not store user funds or sensitive financial data. Instead, it leverages existing bank infrastructure:
 - 🌕 The **Issuer Bank** holds the customer's account.
 - 🌕 The UPI platform, upon receiving a payment instruction from a app, routes the request to the payer's bank for authorization.
 - 🌕 The bank authenticates the transaction using its own systems (e.g., via UPI PIN, which is a secret key that decrypts a transaction token locally on the user's device) and performs the debit.
3. **Virtual Payment Address (VPA):** A key abstraction layer that eliminates the need to share sensitive bank details. A VPA (e.g., `name@psp`) is a human-readable identifier that maps to a user's underlying bank account information. This mapping is stored in a

centralized UPI mapper, which resolves the VPA to the specific bank account and Indian Financial System Code (IFSC) during transaction routing.

4. **Immediate Payment Service (IMPS):** UPI is an application layer that runs on top of the existing IMPS settlement infrastructure. While UPI handles the request, authentication, and messaging, IMPS performs the actual real-time gross settlement (RTGS) between bank accounts, ensuring funds are moved instantly.
5. **API-First Design for Ecosystem Growth:** UPI provides a set of public APIs that allow Third-Party Application Providers (TPAPs like PhonePe, Google Pay) and banks to build their own front-end applications. This fosters innovation and competition on the user interface while the backend protocol remains standardized and interoperable.

In essence, UPI is a **routing and messaging switch** that creates a seamless, single-identifier payments network. It disintermediates the need for a stored-value account by providing a real-time, account-to-account bridge, making it fundamentally different from the closed-loop ledger model of e-wallets. Its security is derived from asymmetric cryptography and the delegation of final authentication to the user's own bank.

E-WALLETS AND UNIFIED PAYMENTS INTERFACE:

1. **E-WALLETS:** The e-wallet system adopts a distributed microservices architecture with event sourcing and CQRS (Command Query Responsibility Segregation) patterns. The system operates on a service mesh topology utilizing Kubernetes orchestration for container management and service discovery.

I. Presentation & Gateway Layer

The user interaction originates at the **Frontend** client. Incoming requests are first processed by a **Rate Limiter** to enforce API consumption policies and mitigate abuse. Subsequently, security middleware, such as **Helmet**, is employed to implement critical HTTP headers, bolstering defenses against common web vulnerabilities. A **Wrapper** service provides an abstraction layer for request normalization and initial validation before dispatching to the core **API Layer**.

II. Application & Integration Layer

The API Layer interfaces with a **Load Balancer** (e.g., Elastic Load Balancer) to distribute incoming traffic efficiently across available backend resources. The system exposes a **GraphQL** endpoint for flexible, client-defined queries, alongside traditional **REST APIs**. All endpoints are fortified with comprehensive protection schemes, including authentication, authorization, and additional rate-limiting. Support for the latest transport security protocols, such as **TLS 1.3**, is mandated for all service-to-service communication.

Application logic is executed on backend **Servers** and **Systems**. Complex, asynchronous tasks are offloaded from the main request/response cycle to dedicated worker processes. This is facilitated by robust queueing systems like **RabbitMQ** or **BullMQ**, which manage job persistence, prioritization, and distribution.

III.. Data & Event Streaming Layer

Performance at the application layer is optimized through extensive use of **Redis**, which serves a dual purpose: as a distributed **caching** layer to reduce latency and database load, and as a high-speed **message broker** for real-time pub/sub functionalities and inter-service communication.

For high-throughput, durable event streaming, the architecture incorporates **Apache Kafka**. This platform acts as the central nervous system for event-driven workflows, functioning as a publish/subscribe service and a permanent log for all system events. It ensures reliable, ordered message delivery to downstream consumers.

IV. Persistence Layer

The final persistence tier employs a database replication strategy designed to optimize for both performance and reliability. Data is written to a single **Primary Database (Read/Write)** to maintain consistency. To scale read capacity and enhance availability, this database is replicated to one or more **Replica Databases (Read-Only)**. A final **Clone Database**, with read/write permissions, is maintained for critical operational purposes, including disaster recovery, analytics, and backup integrity checks without impacting production system performance.

The Backend of the software Architecture includes Authentication and authorization service which like any general authentication service includes the following steps:

1. Identification of the user, if found. Otherwise , the user will have to sign up in the application with all new credentials of the user which gets validated , verified and if the data is safe then the backend receives the data and stores the data in the Read / Write Database and a token may be generated for confirmation and further verification of the user. These tokens act as an identity and verification code for authentication systems. Examples of unique identities that are required , asked for the authentication are username , email , password and pin. The forget password sends a code or link or token or authenticator code for verification purpose.
2. Credential Submission: The user provides one or more credentials to prove their identity. Secured authentication requires at least two of the following factors:
 - Knowledge factor ("Something you know"): Information the user possesses, such as a password, PIN, or security question.
 - Possession factor ("Something you have"): A physical object or device that the user possesses, like a security token, authenticator app, or mobile phone.
 - Inherence factor ("Something you are"): A unique biological trait of the user, such as a fingerprint, facial scan, or voice recognition.

- Location factor ("Somewhere you are"): Verification based on the user's geographic location.
3. Verification or Validation: The user email or user name may behave like a public key and the pin or password is the private key, both of which are encrypted and sent to servers. The system receives the user's credentials and validates them against the information stored in its database.
 - A. Traditional password verification: The system checks the submitted password against a securely stored cryptographic hash (a one-way encrypted version) of the user's actual password. This prevents the server from storing plain-text passwords.
 - B. MFA verification: The system verifies each credential provided. For example, if two-factor authentication (2FA) is used, the system first verifies the password and then sends a unique, one-time code to the user's mobile device for verification.
 4. Authentication decision: Based on the verification result, the system makes an access decision.
 - Success: If the credentials match, the system authenticates the user and grants them access to the requested resources. For web applications, a secure, temporary token is often issued to maintain the user's session without requiring them to re-enter credentials for every action.
 - Failure: If the credentials do not match, authentication is denied, and access is refused. For security, a generic error message is returned to prevent attackers from discovering which specific piece of information was incorrect.
 5. Authorization (Post-authentication): After successful authentication, the system determines what the user is permitted to do based on their assigned permissions or roles. This is a separate step from authentication, as proving identity does not automatically grant universal access.

Role- Based Access Control: RBAC, or Role-Based Access Control, is a method of restricting system access to authorized users based on their job function within an organization. Rather than individually managing permissions for each user, you assign permissions to roles, and then assign roles to users. This simplifies administration, enhances security, and improves compliance.

The RBAC model is built on three core principles:

- **Role assignment:** A user can only gain access to permissions by being assigned to a role.
- **Role authorization:** A user must be authorized to activate or assume a specific role.
- **Permission authorization:** A user can only perform authorized operations based on the permissions granted to their role.

Here is a step-by-step breakdown of how RBAC operates in practice:

1. **Define roles:** Administrators create roles based on job functions or responsibilities within the organization. For example, roles might include "Admin," "Editor," and "Viewer".
2. **Assign permissions to roles:** A set of specific permissions and privileges are assigned to each defined role. For instance:
 - 🟡 **Admin:** Can create, edit, and delete content and manage users.
 - 🟡 **Editor:** Can create and edit content but cannot manage users.
 - 🟡 **Viewer:** Can only view content.
3. **Assign users to roles:** Users are assigned one or more roles based on their job function. The user then inherits the permissions associated with that role. If a user's responsibilities change, their access rights are updated simply by changing their role assignment.
4. **Authorization checks:** When a user attempts to perform an action, the system checks their assigned role(s) to see if they have the necessary permissions. If the role has the permission, access is granted. Otherwise, access is denied.

Types of RBAC models

There are several RBAC models, each building on the core principles:

- **Core RBAC:** The basic model for assigning permissions to roles and roles to users.

- **Hierarchical RBAC:** Introduces role hierarchies, where higher-level roles automatically inherit the permissions of lower-level roles. For example, a senior manager might automatically inherit all the permissions of a manager.
- **Constrained RBAC:** Adds "separation of duties" to prevent conflicts of interest. For example, the same user cannot be assigned conflicting roles, such as the person who submits a request and the person who approves it.
- **Symmetric RBAC:** Includes mechanisms for auditing and reviewing permission-role and user-role assignments, ensuring permissions don't accumulate beyond what is necessary.

Benefits of RBAC

- **Enhanced security:** RBAC enforces the principle of least privilege, meaning users only have the access they absolutely need. This minimizes potential damage from compromised accounts or insider threats.
- **Improved efficiency:** Managing permissions on a role-by-role basis is much simpler than managing individual permissions for each user. This streamlines onboarding, offboarding, and role changes.
- **Greater compliance:** By providing a clear and auditable framework for access control, RBAC helps organizations meet regulatory requirements for data privacy.
- **Better scalability:** As an organization grows, RBAC can easily scale to accommodate new users and roles without increasing administrative complexity.

RBAC is a flexible and widely used model for managing access.

Key Components:

- **Identity Provider Integration:** SAML 2.0 and OAuth 2.0 federation support
- **Token Management:** JWT lifecycle management with refresh token rotation
- **Biometric Authentication:** FIDO2/WebAuthn integration for passwordless authentication
- **Session Management:** Redis-based session storage with configurable TTL

Security Features:

- PBKDF2 password hashing with 100,000+ iterations
- AES-256 encryption for sensitive data at rest
- Certificate-based authentication for API access
- Hardware Security Module (HSM) integration for key management

PAYMENT PROCESSING ENGINE: The payment engine implements the Saga pattern for distributed transaction management, ensuring ACID properties across multiple services and external payment gateways. Proper Routing is done and REST APIs are written to send requests and get responses from the servers of the Payment Processing Engine.

Core Functionality:

- **Transaction Orchestration:** Choreography-based saga implementation
- **Payment Gateway Integration:** Unified payment interface supporting 50+ gateways
- **Settlement Processing:** Real-time gross settlement (RTGS) integration
- **Fraud Detection:** Machine learning-based anomaly detection with real-time scoring

Transaction Flow Architecture:

1. **Pre-authorization:** Account validation and fund reservation
2. **Authorization:** External gateway processing with timeout handling
3. **Capture:** Fund transfer confirmation with idempotency guarantees
4. **Settlement:** Automated reconciliation and settlement processing.

WALLET MANAGEMENT SERVICE: Implements double-entry bookkeeping principles with eventual consistency across distributed ledgers. This supports multi concurrency with ISO 4217 currency code compliance. There are atomic balance updates with optimistic locking. Transaction History with event sourcing for complete audit trails. Configurable velocity limits with time-based windows.

FRAUD DETECTION AND RISK MANAGEMENT: Machine Learning Algorithms help in fraud detection. Data Collection is done then the data is refined and then data is prepared for the machine learning models. Feature analysis and Feature Engineering including real-time Feature Engineering with sliding windows is done before feeding the data into the Machine Learning algorithm model . The machine Learning Algorithm model may be Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, K-Nearest Neighbor, Naive Bayes) for different purposes and clustering for anomaly detection . , neural networks, LSTM may also be required to deal with complex data and reinforcement learning. Detection mechanisms like User Behaviour pattern analysis is done with neural networks, geolocation analysis with IP Geofencing with VPN/ Proxy detection. Browser / signature or IPs of the device analysis and device fingerprinting are also required here. Velocity Checking. Transaction frequency, amount and transaction money monitoring. In the risk-scoring algorithm , we require Bayesian Inference for probability calculation, Ensembling models, A/B testing framework for model Validation .

NOTIFICATION AND COMMUNICATION SERVICE : with the help of Apache Kafka , Websocket, Cloud Messaging service , we can implement reliable message delivery with exactly one semantics and delivery acknowledgements. This may include reliable webhook delivery with retry policies.

DATA ARCHITECTURE MANAGEMENT: These include Polyglot persistence strategy with database per service pattern with appropriate data store selection and PostGreSQL clusters are written for transactional data with ACID requirements, Master-slave replication with automatic failover. Here time based partitioning is done for transactional tables and Point-in-time recovery with 15-minute RPO. The Redis Clusters are required for Session Management , Caching , rate limiting and Redis Sentinel for high availability. We require Sorted sets for leaderboards, hash maps for user sessions. There is AOF with background rewriting for durability. We require MongoDB clusters for document-based data , audit logs, and analytics. Replica sets with automatic failover are present. Compound indexes for query optimization are written. Range based Sharding for horizontal scalability.

SECURITY: The Web application Firewall , ModSecurity with OWASP core rule set, Rate Limiting with exponential Backoffs , Virtual Private Clusters and VPCs with private subnets for

database Layers, TLS 1.3 with Perfect Forward Secrecy is required. OWASP validation with whitelist approach ,Parameterized queries and stored procedures, SQL injection Prevention, XSS protection with Content Security Policies with strict directives, Synchronizer token pattern implementation.AES-256-GCM with HSM-managed keys for Data at rest , TLS 1.3 with ECDHE key exchange for data in transit are generally suggested.RSA-4096 for high-value transactions are in requirements. PKCS#11 compliance with key rotation policies are also done to increase security.

Server Hardening, SQL Injection Prevention and Database Security is very important for security. PCIDSScompliance implementation, GDPR ,PSD 2, AML, KVC are also required for the security purposes.

SCALING: Horizontal Scaling Architecture

Auto-scaling Configuration:

- Kubernetes HPA: CPU/memory-based pod autoscaling
- Cluster Autoscaling: Node-level scaling based on resource demands
- Database Scaling: Read replica scaling with connection pooling
- Cache Scaling: Redis cluster expansion with consistent hashing

Performance Optimization Strategies

Application-Level Optimizations:

- Connection Pooling: HikariCP with optimized pool sizes
- Query Optimization: Database index analysis and query plan optimization
- Caching Strategy: Multi-level caching with cache-aside pattern
- Async Processing: Non-blocking I/O with reactive programming models

Infrastructure Optimizations:

- CDN Integration: Static asset delivery with global edge locations
- Load Balancing: Layer 7 load balancing with health checks

- **Resource Allocation:** JVM tuning with G1 garbage collector
- **Network Optimization:** TCP optimization and keep-alive configuration

Production Environment Architecture

Infrastructure as Code:

- **Terraform Configuration:** Multi-environment infrastructure provisioning
- **Ansible Automation:** Configuration management and deployment automation
- **Docker Containerization:** Immutable infrastructure with security scanning
- **Kubernetes Deployment:** Rolling updates with zero-downtime deployments

CI/CD Pipeline Implementation:

- **GitOps Workflow:** Git-based deployment with ArgoCD
- **Automated Testing:** Unit, integration, and end-to-end test automation
- **Security Scanning:** SAST/DAST integration with vulnerability assessment
- **Quality Gates:** Code coverage thresholds and performance benchmarks

9.2 Operational Excellence

Site Reliability Engineering (SRE) Practices:

- **Error Budgets:** Service-level objective management with error budget tracking
- **Incident Response:** On-call rotation with escalation procedures
- **Post-mortem Culture:** Blameless post-incident reviews with action items
- **Capacity Planning:** Predictive scaling based on historical usage patterns

Security Operations Center (SOC):

- **24/7 Monitoring:** Continuous security monitoring with SIEM integration
- **Threat Intelligence:** Real-time threat feed integration with automated response
- **Incident Response:** Automated security incident containment and remediation
- **Compliance Monitoring:** Continuous compliance validation with audit trails

UNIFIED PAYMENTS INTERFACE:

I'll provide a detailed explanation of the UPI (Unified Payments Interface) architecture, focusing on its unique technical implementation and how it differs from traditional e-wallet systems.

UPI Architecture: Technical Deep Dive

Core Architectural Philosophy

UPI operates fundamentally differently from e-wallet systems - it's **not a payment system but a payment protocol**. Unlike e-wallets that maintain closed-loop ledgers with stored value, UPI functions as a **real-time interoperable switching network** that facilitates direct bank-to-bank transfers without intermediate value storage.

1. Protocol-Based Architecture vs. Application Architecture

UPI as Digital Public Infrastructure (DPI)

UPI is architected as **open-source digital public infrastructure**, meaning:

Technical Implementation:

- **Open API Standards:** Publicly available API specifications that any entity can implement
- **Protocol Compliance:** Standardized message formats, authentication mechanisms, and transaction flows
- **Interoperability Framework:** Any compliant application can interact with any participating bank
- **Network Effect:** Value increases exponentially with each new participant

Contrast with E-wallet:

- E-wallets: Proprietary, closed systems with custom APIs

- UPI: Open protocol with standardized interfaces across all participants

Decentralized vs. Centralized Processing

UPI's Decentralized Model:

User App → UPI Switch (NPCI) → Payer Bank → IMPS → Payee Bank → UPI Switch → Recipient App

Key Technical Difference:

- **No Central Ledger:** UPI doesn't store transaction balances or user funds
- **Routing Function:** Acts as a message router and transaction coordinator
- **Bank Authentication:** Final transaction authorization happens at individual banks
- **Real-time Settlement:** Immediate fund movement through existing banking infrastructure

2. UPI Switch Architecture: The Central Nervous System

National Payments Corporation of India (NPCI) Switch

The UPI switch operates as a **high-availability, real-time message processing system:**

Technical Specifications:

- **Message Processing Capacity:** 100,000+ transactions per second
- **Availability:** 99.99% uptime with redundant data centers
- **Latency:** Sub-second transaction processing and routing
- **Protocol Support:** Multiple message formats (ISO 20022, custom UPI messages)

Switch Functions:

UPI Switch Core Functions

message_routing:

- source_validation
- destination_resolution
- protocol_translation
- load_balancing

transaction_coordination:

- two_phase_commit
- timeout_handling
- error_recovery
- idempotency_management

compliance_enforcement:

- regulatory_validation
- risk_scoring
- fraud_detection
- audit_logging

Message Flow Architecture

1. Request Initiation:

```
{  
  
  "messageId": "HDFC0001234567890123456789012345",  
  
  "timestamp": "2024-09-15T10:30:00.000Z",  
  
  "txnId": "TXN123456789012345",  
  
  "payerVPA": "user@paytm",  
  
  "payeeVPA": "merchant@phonepe",  
  
  "amount": "100.00",  
  
  "currency": "INR",  
  
  "note": "Payment for goods"  
}
```

2. Switch Processing:

- **VPA Resolution:** Map virtual addresses to actual bank accounts
- **Routing Decision:** Determine payer and payee banks
- **Protocol Translation:** Convert between different bank message formats
- **Risk Assessment:** Real-time fraud and compliance checking

3. Bank Integration:

- **Payer Bank:** Authenticate user, validate balance, reserve funds
- **Payee Bank:** Validate account, prepare for credit
- **Settlement:** IMPS-based real-time gross settlement

3. Virtual Payment Address (VPA): Abstraction Layer

Technical Implementation

VPA serves as a **cryptographic abstraction layer** that hides sensitive banking information:

VPA Structure:

user.identifier@payment.service.provider

|—— User Identifier (chosen by user)

|—— @ (separator)

|—— PSP Handle (registered payment service provider)

Backend Mapping:

```
{  
  
  "vpa": "john.doe@paytm",  
  
  "mapping": {  
  
    "account_number": "123456789012",  
  
    "ifsc_code": "HDFC0001234",  
  
    "bank_name": "HDFC Bank",  
  
    "account_holder": "John Doe",  
  
    "mobile_number": "+919876543210"  
  
  },  
  
  "encryption": "AES256",  
  
  "last_updated": "2024-09-15T08:00:00.000Z"  
  
}
```

VPA Resolution Process

Centralized Mapper Service:

- **Directory Service:** Maintains VPA-to-account mappings for all registered users
- **High Availability:** Replicated across multiple data centers
- **Caching Strategy:** Distributed caching for frequently accessed VPAs
- **Security:** Encrypted storage with access logging

Resolution Flow:

1. **VPA Lookup:** Switch queries central mapper for account details
2. **Bank Identification:** Extract bank routing information from IFSC
3. **Account Validation:** Verify account status and holder details
4. **Response Caching:** Cache valid mappings for performance optimization

4. Authentication & Authorization Architecture

Multi-layered Security Model

1. Device-Level Security:

// Example UPI PIN generation

```
public class UPIPinGenerator {
```

```
    public String generateEncryptedPin(String userPin, String deviceId) {
```

```
        // Device-specific encryption key
```

```
        SecretKey deviceKey = deriveDeviceKey(deviceId);
```

```
        // Combine PIN with device signature
```

```

String pinWithDeviceSignature = userPin + generateDeviceSignature();

// Encrypt using AES-256

return encryptAES256(pinWithDeviceSignature, deviceKey);

}

}

```

2. Transaction Token System:

- **Dynamic Token Generation:** Each transaction gets a unique cryptographic token
- **Time-bound Validity:** Tokens expire within 30 seconds of generation
- **Device Binding:** Tokens are cryptographically bound to specific devices
- **Replay Protection:** Used tokens are immediately invalidated

3. Bank-Level Authentication:

-- Example bank authentication flow

```

SELECT account_status, available_balance, daily_limit

FROM customer_accounts ca

JOIN transaction_limits tl ON ca.account_id = tl.account_id

WHERE ca.account_number = ?

AND ca.mobile_number = ?

AND decrypt_upi_pin(ca.encrypted_pin, ?) = ?;

```

Cryptographic Implementation

Asymmetric Key Infrastructure:

- **RSA-2048**: Used for initial key exchange and digital signatures
- **ECDSA**: Elliptic curve signatures for mobile device constraints
- **AES-256**: Symmetric encryption for bulk data protection
- **HMAC-SHA256**: Message authentication and integrity verification

Key Management:

key_hierarchy:

root_ca:

algorithm: RSA-4096

usage: Certificate signing

rotation: Annual

bank_certificates:

algorithm: RSA-2048

usage: Transaction signing

rotation: Bi-annual

device_keys:

algorithm: ECDSA P-256

usage: Transaction authorization

rotation: Per-app-install

5. IMPS Integration: Settlement Layer

Immediate Payment Service (IMPS) Architecture

UPI leverages existing IMPS infrastructure for actual fund settlement:

IMPS Technical Specifications:

- **Operating Hours:** 24x7x365 availability
- **Settlement Model:** Real-Time Gross Settlement (RTGS)
- **Transaction Limits:** Up to ₹5 lakh per transaction
- **Processing Time:** Typically under 10 seconds

Integration Architecture:

UPI Layer: [Message Routing] → [Authentication] → [Protocol Translation]

↓

IMPS Layer: [Fund Reservation] → [Settlement] → [Confirmation]

↓

Banking Layer: [Account Debit] → [RTGS Transfer] → [Account Credit]

Settlement Process Flow

1. Pre-settlement Validation:

```
def validate_settlement_request(transaction):
```



```

validations = [

    validate_account_status(transaction.payer_account),

    validate_available_balance(transaction.payer_account, transaction.amount),

    validate_transaction_limits(transaction.payer_account, transaction.amount),

    validate_regulatory_compliance(transaction)

]

return all(validations)

```

2. Two-Phase Commit Protocol:

- **Phase 1 (Prepare):** Reserve funds in payer account, validate payee account
- **Phase 2 (Commit):** Execute actual fund transfer via IMPS
- **Rollback:** Release reserved funds if any phase fails

3. Settlement Confirmation:

- **Immediate Acknowledgment:** Transaction success/failure notification
- **Reconciliation:** End-of-day settlement file generation
- **Audit Trail:** Complete transaction logging for regulatory compliance

6. Third-Party Application Provider (TPAP) Integration

API-First Ecosystem Design

Standardized API Specifications:

UPI API Endpoint Structure

base_url: <https://api.upi.npci.org/v2>

endpoints:

collect_request:

path: /collect

method: POST

authentication: Bearer token

pay_request:

path: /pay

method: POST

authentication: mTLS + Bearer token

transaction_status:

path: /status/{txnId}

method: GET

authentication: API key

TPAP Certification Process:

1. **Technical Integration:** API compliance testing
2. **Security Assessment:** Penetration testing and security audit
3. **Operational Readiness:** Load testing and disaster recovery validation
4. **Regulatory Compliance:** AML/KYC procedure verification

5. **Go-Live Certification:** Final approval from NPCI

SDK and Integration Framework

UPI SDK Architecture:

// Example UPI SDK structure

```
public class UpiSdk {  
  
    private final AuthenticationManager authManager;  
  
    private final MessageRouter messageRouter;  
  
    private final SecurityProvider securityProvider;  
  
  
    public TransactionResult initiatePayment(PaymentRequest request) {  
  
        // Validate request  
  
        validateRequest(request);  
  
  
        // Generate transaction token  
  
        String txnToken = securityProvider.generateTransactionToken();  
  
  
        // Route to UPI switch  
  
        UpiMessage message = createUpiMessage(request, txnToken);  
  
        return messageRouter.routeMessage(message);  
  
    }  
}
```

}

7. Real-time Processing & Performance Architecture

High-Throughput Message Processing

Kafka-based Event Streaming:

kafka_configuration:

topics:

transaction_requests:

partitions: 50

replication_factor: 3

retention: 7 days

settlement_events:

partitions: 20

replication_factor: 3

retention: 30 days

consumers:

transaction_processor:

instances: 10

max_poll_records: 100

processing_guarantee: exactly_once

Load Balancing Strategy:

- **Geographic Distribution:** Switch nodes across multiple cities
- **Bank-wise Partitioning:** Route requests based on bank capacity
- **Time-based Sharding:** Distribute load across processing windows
- **Dynamic Scaling:** Auto-scaling based on transaction volume

Performance Optimization Techniques

1. Connection Pooling:

// Bank connection management

@Configuration

public class BankConnectionConfig {

 @Bean

 public HikariDataSource bankConnectionPool() {

 HikariConfig config = new HikariConfig();

 config.setMaximumPoolSize(100);

 config.setMinimumIdle(10);

 config.setConnectionTimeout(5000);

 config.setIdleTimeout(300000);

 return new HikariDataSource(config);

```
}  
  
}
```

2. Caching Architecture:

- **VPA Resolution Cache:** Redis cluster for frequently accessed VPAs
- **Bank Routing Cache:** Pre-computed routing tables for IFSC codes
- **Transaction Deduplication:** Bloom filters for duplicate detection
- **Rate Limiting Cache:** Sliding window counters for API throttling

8. Regulatory Compliance & Risk Management

Real-time Fraud Detection

Machine Learning Pipeline:

Example fraud detection model

```
class UPIFraudDetector:
```

```
    def __init__(self):
```

```
        self.velocity_checker = VelocityChecker()
```

```
        self.behavioral_analyzer = BehaviorAnalyzer()
```

```
        self.network_analyzer = NetworkAnalyzer()
```

```
    def assess_transaction_risk(self, transaction):
```

```
        risk_factors = {
```

```
            'velocity_risk': self.velocity_checker.check(transaction),
```

```

        'behavioral_risk': self.behavioral_analyzer.analyze(transaction),

        'network_risk': self.network_analyzer.evaluate(transaction)

    }

```

```

risk_score = self.calculate_composite_risk(risk_factors)

return self.determine_action(risk_score)

```

Risk Assessment Factors:

- **Velocity Analysis:** Transaction frequency and amount patterns
- **Geolocation Verification:** Device location vs. historical patterns
- **Network Analysis:** IP reputation and proxy detection
- **Behavioral Biometrics:** Typing patterns and device interaction analysis

Regulatory Reporting

Automated Compliance Reporting:

-- Example regulatory report generation

```
CREATE VIEW daily_transaction_report AS
```

```
SELECT
```

```
    DATE(transaction_timestamp) as report_date,
```

```
    COUNT(*) as total_transactions,
```

```
    SUM(amount) as total_volume,
```

```
    COUNT(DISTINCT payer_vpa) as unique_payers,
```

```
COUNT(CASE WHEN amount > 200000 THEN 1 END) as high_value_transactions  
  
FROM upi_transactions  
  
WHERE transaction_status = 'SUCCESS'  
  
GROUP BY DATE(transaction_timestamp);
```

Compliance Features:

- **Transaction Monitoring:** Real-time suspicious activity detection
- **AML Reporting:** Automated generation of Suspicious Transaction Reports (STR)
- **KYC Integration:** Customer verification status tracking
- **Audit Trails:** Immutable transaction logs for regulatory inspection

9. Disaster Recovery & Business Continuity

Multi-datacenter Architecture

Geographic Distribution:

datacenters:

primary:

location: Mumbai

role: Active processing

capacity: 60% of total load

secondary:

location: Chennai

role: Active-passive backup

capacity: 40% of total load

disaster_recovery:

location: Delhi

role: Cold standby

rto: 15 minutes

rpo: 30 seconds

Failover Mechanisms:

- **Automatic Failover:** Health check-based switching between datacenters
- **Database Replication:** Synchronous replication for critical data
- **Message Queue Replication:** Cross-datacenter message mirroring
- **DNS-based Routing:** Geographic DNS resolution for disaster recovery

Business Continuity Planning

Recovery Time Objectives (RTO):

- **Critical Services:** < 15 minutes (VPA resolution, transaction routing)
- **Settlement Services:** < 5 minutes (IMPS integration)
- **Reporting Services:** < 1 hour (compliance and analytics)
- **Full System Recovery:** < 30 minutes (complete platform restoration)

This UPI architecture demonstrates how a protocol-based approach can create a more resilient, scalable, and interoperable payment ecosystem compared to traditional application-centric

e-wallet systems. The key innovation lies in UPI's role as infrastructure rather than application, enabling a truly open and collaborative payments ecosystem.

CONCLUSION:

The evolution of electronic payment systems represents a fundamental shift in global financial infrastructure, moving from closed, proprietary platforms toward open, interoperable networks. E-wallets, as stored-value systems, demonstrated the potential for digital convenience by creating curated, user-friendly ecosystems. However, the advent of protocols like the Unified Payments Interface (UPI) marks a paradigm shift by reimagining payments not as a product but as a public utility. UPI's architecture—built on open standards, real-time bank settlement, and a decentralized model—has proven superior in driving financial inclusion, fostering innovation, and achieving massive scale at low cost.

The future of digital payments lies in this infrastructure model, where interoperability, security, and accessibility are foundational. While e-wallets will continue to serve niche markets requiring controlled user experiences, the dominant trajectory is toward unified, account-agnostic networks that prioritize public benefit over private control. The success of such systems hinges on robust technical governance, relentless focus on security and privacy, and deep collaboration between regulators, financial institutions, and technology providers. Ultimately, the goal is no longer merely facilitating digital transactions, but creating an inclusive, efficient, and resilient economic backbone for the digital age.

REFERENCES:

1. https://en.wikipedia.org/wiki/Digital_wallet
2. <https://dev.to/zeeshanali0704/system-design-upi-unified-payment-interface-2ng3>